

GeoRust

Gruppo 8

Samuele Accio
Raul Oppezzo
Stefano Tallone
Giulio Titonel



Introduzione a GeoRust

Progetto

Applicazione client/server per gestire geolocalizzazione e comunicazione di una flotta di veicoli

Server

Il server autentica gli utenti, analizza il movimento e gestisce i messaggi

Client

Ogni client simula un veicolo: legge un percorso e invia la posizione ogni 30 secondi

Stack tecnologico

Tutto scritto in Rust; Tokio coordina le attività concorrenti su entrambi i lati

Requisiti



<u>Registrazione sicura</u>	Account e password, hash Argon2 con salt casuale
<u>Posizione ogni 30 secondi</u>	Il client invia coordinate lette da un percorso CSV
<u>Stati utente</u>	Disconnected · Moving · Still, con soglia di 3 minuti
<u>Analisi del movimento</u>	Distanza, tempo di moto e di pausa, velocità media
<u>Messaggistica</u>	Messaggi diretti e broadcast tra server e client
<u>Monitoraggio CPU</u>	Log del consumo del processo ogni 120 secondi

Un workspace, tre crate

common

Libreria con tipi condivisi:

- Messaggi
- Stati
- Coordinate

client

- Autenticazione via REST
- Lettura e validazione percorsi CSV
- Simulazione temporale del movimento
- Interfaccia testuale (Ratatui)

server

- API REST + endpoint WebSocket
- Macchina a stati e statistiche
- Persistenza SQLite (SQLx)
- Console admin (Ratatui)



Due fasi di **comunicazione**

1. REST

- **POST/register** – crea l'utente, password come hash Argon2
- **POST/login** – verifica le credenziali
- Il server genera un token e lo restituisce al client
- Dopo la registrazione, il login parte automaticamente

2. WebSocket

- **GET/ws** con il token nell'header Authorization: Bearer
- Connessione persistente e bidirezionale
- Trasporta posizioni, messaggi ed esiti del tragitto
- Messaggi tipizzati in **common**, serializzati in JSON

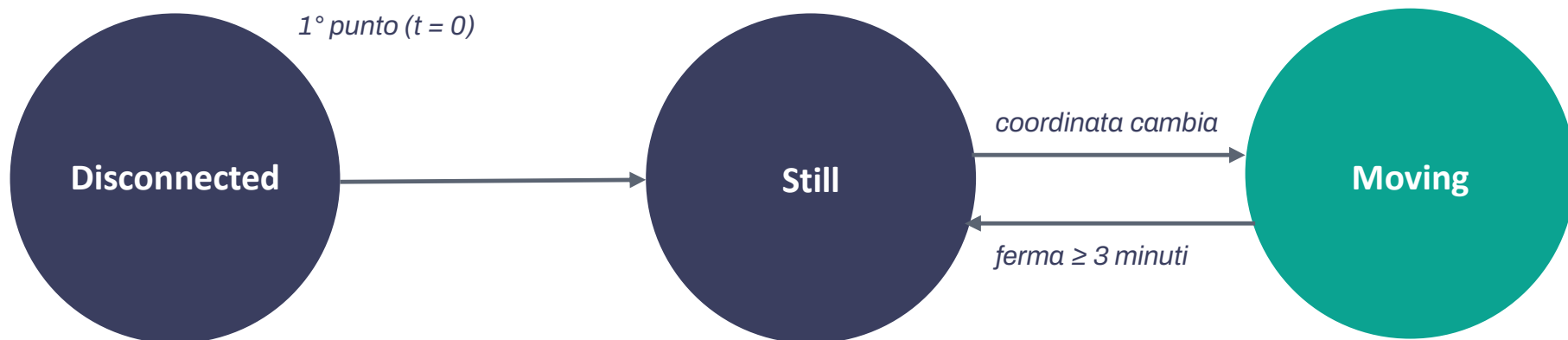


Simulazione del movimento

- Simulazione del movimento tramite file CSV. Colonne: time, latitude, longitude
- Il primo punto ha tempo 00:00, ogni successivo è a +30s esatti
- Coordinate ripetute e consecutive rappresentano una sosta
- Il client valida l'intero percorso prima di partire; il server verifica nuovamente i tempi ricevuti
- Nel client, è stata aggiunta la possibilità di accelerare il tempo di simulazione. I tempi logici restano invariati



La macchina a stati del tragitto



- **Still**, coordinata invariata → resta **Still**, +30s al tempo di pausa
- **Still**, coordinata cambia → passa a **Moving**, +30s al tempo di movimento
- **Moving**, ferma per meno di 3 minuti → il tempo resta "in sospeso"
- **Moving**, ferma per almeno 3 minuti → passa a **Still**, l'intero periodo va in pausa

Il modello dati in SQLite



<u>users</u>	Identificativo, username, hash della password e colonna current_state.
<u>trips</u>	Utente, data del tragitto, distanza totale, secondi di movimento e secondi di pausa.
<u>trips_point</u>	Indice del punto, tempo logico e coordinate associate a un tragitto.
<u>messages</u>	Mittente, destinatario, tipo, contenuto, data di creazione e stato di lettura del messaggio

Al termine di un tragitto, dati aggregati e punti vengono salvati in un'unica transazione: tutto o niente, nessun tragitto parziale nel database.

Dopo un salvataggio riuscito il tragitto viene rimosso dalla memoria. I punti restano disponibili nel database per una futura ricostruzione del percorso, anche se l'attuale console amministrativa utilizza soltanto i valori aggregati presenti in trips. Un tragitto interrotto senza il messaggio di completamento viene invece eliminato dalla memoria senza essere salvato.

Analisi del movimento



La console amministrativa permette di interrogare le statistiche di uno specifico utente per il **giorno**, la **settimana** o il **mese** corrente. Gli intervalli sono determinati a partire dalla data UTC del server: per la settimana viene considerato come inizio il lunedì, mentre per il mese viene utilizzato il primo giorno. Le query sommano la **distanza**, il **tempo di movimento** e il **tempo di pausa** dei tragitti compresi nel periodo selezionato.

La velocità media viene calcolata dividendo la distanza complessiva per il solo tempo trascorso in movimento e convertendo il risultato in chilometri orari. Le pause non entrano quindi nel denominatore; se non è presente alcun tempo di movimento, il valore restituito è zero.

Le query vengono fatte sulla tabella **trips**, che contiene già i dati in forma aggregata. Il calcolo dei dati in forma aggregata, quali distanza totale, tempo in movimento e tempo in pausa avviene in corrispondenza del completamento di un tragitto da parte del client

Canali **Tokio** per la comunicazione



- Ogni connessione WebSocket ha un canale **mpsc** dedicato (messaggi diretti)
- Un canale broadcast condiviso raggiunge tutti gli utenti connessi
- **tokio::select!** ascolta contemporaneamente socket, canale diretto e broadcast
- Tutti i messaggi sono persistiti in SQLite, con stato di lettura
- Messaggi diretti pendenti recapitati alla riconnessione; i broadcast non vengono ripetuti

Comandi testuali con Ratatui

Client

```
GeoRust Client

Accesso
  Autenticazione richiesta
  Per continuare devi effettuare il login oppure registrarti.
  Digita 'l' / 'login' oppure 'r' / 'register'.
  Operazione: Non selezionata
  Username: -
  Scegli login oppure registrazione.
  Dopo la registrazione verrà effettuato automaticamente il login.

Login [l] o Registrazione [r]
> |

GeoRust User Console user1 (#1): CONNESSO
Comandi disponibili
  help: guida  msg <testo>: messaggio al server  start: nuovo trip  exit: chiudi il client

Console
  Autenticazione completata.
  Connessione al server in corso...
  Connessione WebSocket attiva.

Messaggi
  [12:46:10] [Broadcast] hello

Comando
> |
```

Comandi testuali con Ratatui

Server

```
GeoRust Admin Dashboard
Comandi disponibili
help: guida  users: utenti registrati  logs <username>: ultimi 10 messaggi dell'utente  stats <username>: analisi
viaggi  msg: invia messaggi singoli o broadcast  exit: chiudi il server

Console
Avvio del server in corso...
Server avviato con successo. In ascolto sulla porta 3000.
Digita 'help' per vedere la lista dei comandi.
> msg
Scegli il tipo di messaggio:
  1. Diretto a un utente
  2. Broadcast (a tutti)
> 2
Inserisci il testo del messaggio broadcast:
> hello
Broadcast inviato con successo!

Messaggi
[09:30:26] [Broadcast] hello world!
[09:31:17] [Da: user1] hello to you!
[11:56:36] [A: user1] ciao!
[15:44:32] [Broadcast] msg2
[17:33:58] [A: user2] msg3
[17:50:04] [Broadcast] broad
[12:46:10] [Broadcast] hello

Online users (1)
● #1 - user1

Comando
> |
```

Test e CI multiplatform



- Test automatici distribuiti nei tre crate per verificare soprattutto le parti che contengono regole applicative.
- Pipeline GitHub Actions su **windows-latest** e **ubuntu-latest**
- Cache delle dipendenze e della cartella target per build più rapide
- Non sono presenti test di integrazione che avviano realmente server e client come processi separati. Le interazioni complete, come la riconnessione, il rifiuto delle connessioni duplicate e la consegna dei messaggi tra più client, richiedono quindi anche una verifica manuale.

CPU log e dimensioni

- Task dedicato: ogni 120s legge l'uso CPU del processo via **sysinfo**
- Log in logs/cpu_usage_log.txt: PID, % istantanea, uptime, stima cumulativa
- La stima cumulativa NON è un valore esatto del sistema operativo
- Operazioni di rete e DB asincrone: una connessione in attesa non blocca le altre
- Pool **SQLite** fino a 50 connessioni; canali interni a capacità limitata

3,4 MB
client.exe

6,5 MB
server.exe





Grazie!

GeoRust – Gruppo 8

Samuele Accio – Raul Oppezzo – Stefano Tallone – Giulio Titonel